Universidad Politécnica De Atlautla

“Fundamentos de Inteligencia Artificial”

(Sección 4 de 6 – 2.4 Algoritmos de búsqueda ciega (BFS / DFS))

Segundo Parcial

Docente: García Lara Javier

Alumna: Sanchez Fragozo Dulce Maria - 24305056

Carrera: Ingeniería en Tecnologías de la Información e Innovación Digital.

Grupo 171



23 de Octubre del 2026, Atlautla de Victoria, Edomex

1. Planteamiento del problema

Dado un laberinto representado como una cuadrícula de 11×14 celdas, se busca un camino desde la celda de inicio (arriba a la izquierda) hasta la meta (fila 10, columna 10) sin atravesar paredes. Se trata de un problema de **búsqueda ciega**: los algoritmos no usan información sobre la distancia a la meta, solo la estructura del espacio de estados (Russell & Norvig, 2020).

2. Formulación del problema

Componente	Definición
Estados	Cada celda libre (fila, columna). Hay 87 estados alcanzables en total.
Estado inicial	(0, 0)
Prueba de meta	El estado actual es (10, 10)
Acciones	Mover arriba, derecha, abajo o izquierda si la celda destino existe y no es pared
Modelo de transición	Resultado(s, a) = celda vecina en la dirección a
Costo de paso	1 por movimiento (el costo del camino es su número de pasos)

3. Datos y representación del grafo

El laberinto principal es una cuadrícula de 11×14 (0 = libre, 1 = pared). Se modeló como un grafo no dirigido con NetworkX (Hagberg et al., 2008) donde cada celda libre es un nodo y cada par de celdas adyacentes libres es una arista: |V| = 87 y |E| = 89. Como |E| es casi igual a |V|, el laberinto tiene muy pocos ciclos (es casi un árbol), lo que limita las diferencias entre algoritmos.

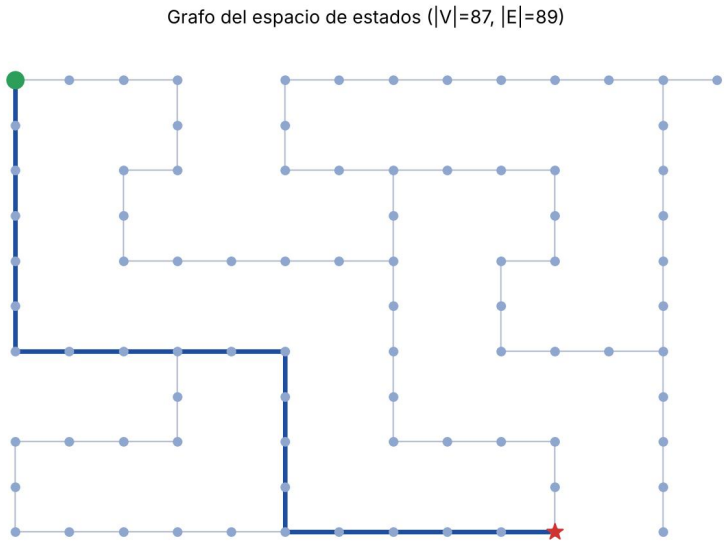


Figura 1. Grafo del espacio de estados; en azul, el camino encontrado por BFS.

4. Procedimiento

- **BFS (búsqueda en anchura).** Usa una cola FIFO: expande todos los nodos a profundidad d antes de pasar a d+1. Marca los estados al generarlos y guarda el padre de cada uno para reconstruir el camino.
- **DFS (búsqueda en profundidad).** Usa una pila LIFO: avanza por una rama hasta que no puede continuar y entonces retrocede. Guarda el padre al sacar el nodo de la pila y mantiene el conjunto de estados visitados para evitar ciclos.

Ambos se ejecutaron sobre el mismo laberinto. Se registró: nodos expandidos, longitud del camino, tamaño máximo de la frontera y tiempo (promedio de 200 corridas). Además se corrió un experimento de escalabilidad con laberintos aleatorios de 10×10 a 60×60 (25 % de paredes, 8 semillas por tamaño, promedio).

5. Resultados

Métrica	**BFS**	**DFS**
Nodos expandidos	64	57
Longitud del camino (pasos)	20	22
Tamaño máximo de la frontera	6	6
Tiempo medio (ms)	0.065	0.066

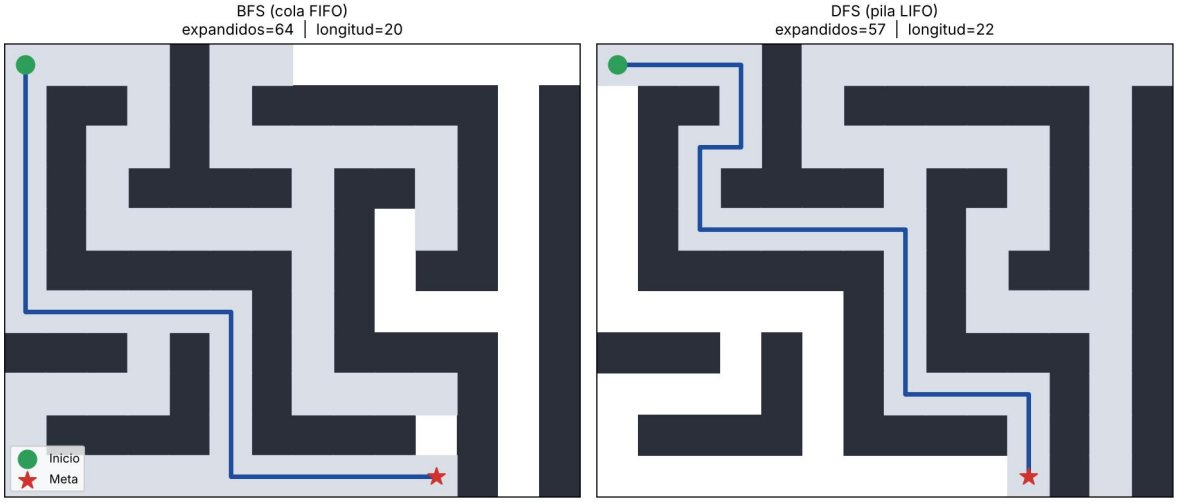


Figura 2. Exploración y camino de BFS (izquierda) y DFS (derecha). Las celdas grises fueron expandidas.

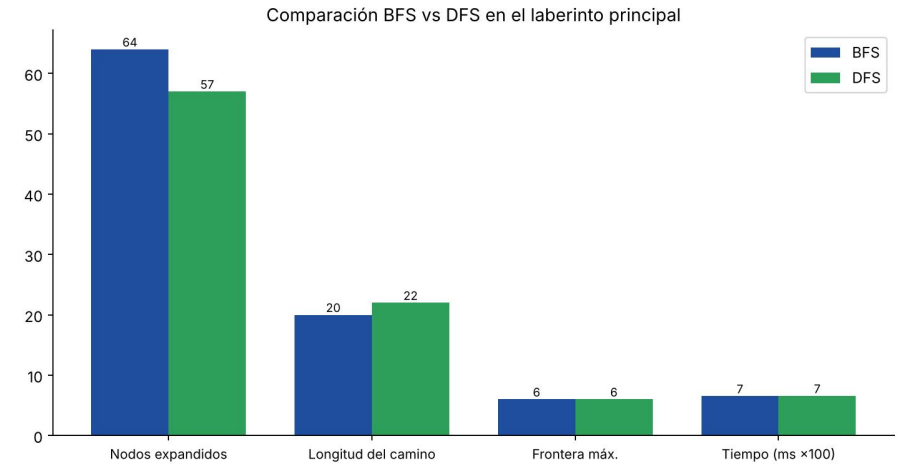


Figura 3. Comparación de métricas (el tiempo se multiplicó ×100 para poder graficarlo en la misma escala).

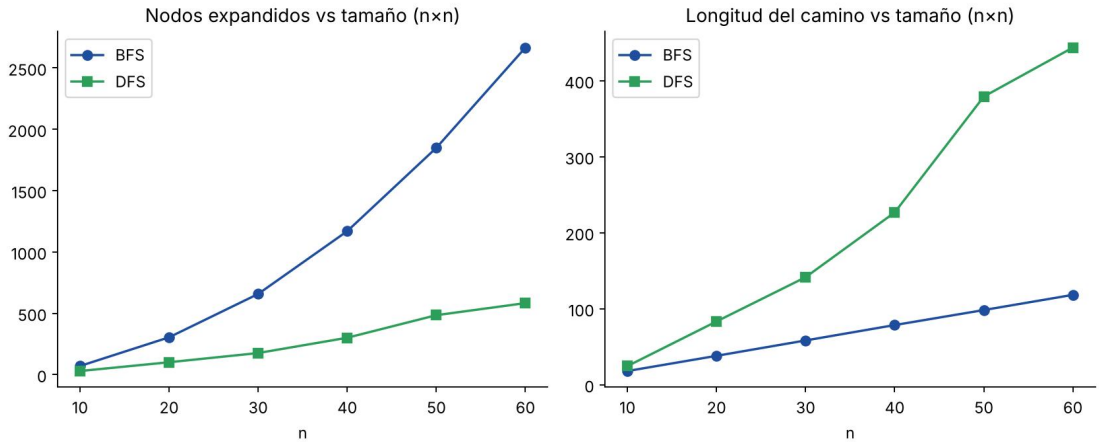


Figura 4. Escalabilidad en laberintos aleatorios n×n (promedio de 8 casos por tamaño).

n	BFS: expandidos	DFS: expandidos	BFS: longitud	DFS: longitud
10	70	29	18	25
20	304	101	38	83
30	658	176	58	142
40	1170	301	79	227
50	1849	485	98	379
60	2667	583	118	444

Simulación animada

Al ejecutar python busqueda_ciega.py se abre una ventana con la animación paso a paso de BFS (izquierda) y DFS (derecha) sobre el mismo laberinto; con un clic en la ventana se pausa y se reanuda. Amarillo = frontera, azul claro = visitado, rojo = nodo que se expande, azul fuerte = camino. La animación también se guarda como resultados_s4/simulacion_bfs_dfs.gif (opciones: --sin-ventana, --velocidad 2). Se aprecia que BFS avanza en "ondas" y DFS se mete por un pasillo hasta el fondo.

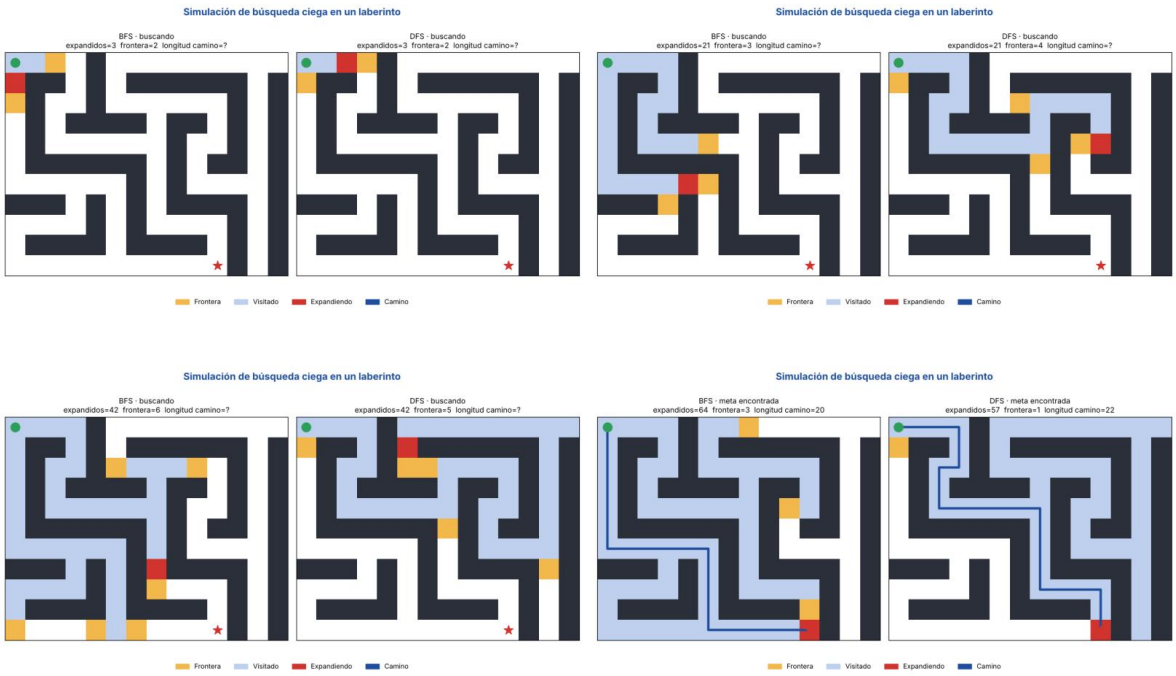


Figura 5. Cuatro fotogramas de la simulación (inicio, 1/3, 2/3 y final).

6. Análisis de complejidad

	BFS	**DFS**
Tiempo	O(b^d) en general; en el grafo, O(V + E)	O(b^m) en general; en el grafo, O(V + E)
Espacio	O(b^d): guarda toda la frontera	O(b·m): solo el camino actual (sin lista de visitados)
Completo	Sí (con factor de ramificación finito)	Sí en grafos finitos con control de visitados
Óptimo	Sí cuando todos los pasos cuestan 1	No

Donde b es el factor de ramificación, d la profundidad de la solución más superficial y m la profundidad máxima. En una cuadrícula b ≤ 3 (sin contar el regreso), por lo que la frontera se mantiene pequeña; en este laberinto fue de 6 nodos para ambos algoritmos (Bohm, 2021; Russell & Norvig, 2020).

7. Conclusiones

- **Optimabilidad.** BFS encontró el camino más corto (20 pasos); DFS encontró uno de 22 pasos. DFS no garantiza optimalidad: devuelve el primer camino que encuentra en su recorrido.
- **Esfuerzo de búsqueda.** En el laberinto principal DFS expandió menos nodos (57 frente a 64), pero fue por la ubicación de la meta; no es una regla general. En los laberintos aleatorios sí se repite: DFS expandió aproximadamente entre 4 y 5 veces

menos nodos que BFS en 60×60 (583 frente a 2667), pero a costa de caminos mucho más largos (444 frente a 118 pasos).

- **Tiempo.** Las diferencias de tiempo en el laberinto principal son del orden de microsegundos y no son concluyentes; el número de nodos expandidos es la métrica más informativa.
- **Idoneidad según el problema.** BFS es preferible cuando importa la solución más corta y la memoria no es limitante (rutas, puzzles de pocos movimientos). DFS conviene cuando cualquier solución sirve y la memoria es crítica o el árbol es muy profundo.
- **Limitación.** Los resultados dependen del laberinto y del orden de los movimientos (arriba, derecha, abajo, izquierda), que afecta sobre todo a DFS.

Declaración de uso de inteligencia artificial

Elemento	Detalle
Herramienta	Claude (Anthropic, 2026), modelo Claude Sonnet 5.5 [modelo de lenguaje de gran tamaño]
Prompt	Se pidió ayuda para resolver la Actividad Parcial 2, Sección 4 (BFS/DFS) a partir de la hoja de instrucciones de la asignatura.
Uso	Generación del borrador del código Python, de las gráficas, de la simulación animada y de la estructura del reporte (Anthropic, 2026).
Verificación	Se ejecutó el código y se revisaron los resultados: BFS obtuvo la longitud mínima (20 pasos) y DFS un camino válido. Las cifras del reporte provienen de metricas.json generado por la ejecución.

Referencias (APA 7.ª ed.)

Anthropic. (2026). *Claude Sonnet 5.5* [Modelo de lenguaje de gran tamaño]. https://claude.ai

Bohm, R. A. (2021). *The fundamentals of search algorithms*. Nova Science.

Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. En G. Varoquaux, T. Vaught, & J. Millman (Eds.), *Proceedings of the 7th Python in Science Conference* (pp. 11–15).

Russell, S., & Norvig, P. (2020). *Inteligencia artificial: Un enfoque moderno* (4.ª ed.). Pearson.

Anexo. Código fuente

busqueda_ciega.py

```
"""
Sección 4 - Algoritmos de búsqueda ciega (BFS / DFS)
Asignatura: Fundamentos de Inteligencia Artificial - UPA
Problema: resolver un laberinto en cuadrícula (estado = celda, acción = mover
arriba/abajo/izquierda/derecha, costo = 1 por paso).

Uso:  python busqueda_ciega.py
Genera: resultados_s4/*.png, resultados_s4/metricas.json y resultados_s4/simulacion_bfs_dfs.gif

SIMULACIÓN: al ejecutar se abre una ventana con la animación paso a paso de BFS y DFS
(clic sobre la ventana = pausar / reanudar). Opciones:
  python busqueda_ciega.py --sin-ventana      (solo guarda el GIF, útil sin pantalla)
  python busqueda_ciega.py --velocidad 3      (más rápido; 0.5 = más lento)
"""
import argparse
import json
import os
import time
from collections import deque

import matplotlib
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation, PillowWriter
from matplotlib.colors import ListedColormap
import networkx as nx
import numpy as np

SALIDA = "resultados_s4"
os.makedirs(SALIDA, exist_ok=True)

# Laberinto principal: 0 = libre, 1 = pared, S = inicio, G = meta
LABERINTO = [
    "S00010000000000",
    "0110101111110",
    "0100100000010",
    "0101111011010",
    "0100000010010",
    "0111111010110",
    "0000001010000",
    "1110101011110",
    "0000101000010",
    "0111101111010",
    "0000000000G10",
]
# Se normaliza a ancho constante (las filas tienen 13 o 14 caracteres)
ANCHO = max(len(f) for f in LABERINTO)
LABERINTO = [f.ljust(ANCHO, "1") for f in LABERINTO]

MOVS = [(-1, 0), (0, 1), (1, 0), (0, -1)]  # arriba, derecha, abajo, izquierda


def cargar(laberinto):
    rejilla = np.zeros((len(laberinto), len(laberinto[0])), dtype=int)
    inicio = meta = None
    for r, fila in enumerate(laberinto):
        for c, ch in enumerate(fila):
            if ch == "1":
                rejilla[r, c] = 1
            elif ch == "S":
                inicio = (r, c)
            elif ch == "G":
                meta = (r, c)
    return rejilla, inicio, meta


def vecinos(rejilla, estado):
    """Generador de estados sucesores (acciones legales)."""
    r, c = estado
    for dr, dc in MOVS:
        nr, nc = r + dr, c + dc
        if 0 <= nr < rejilla.shape[0] and 0 <= nc < rejilla.shape[1] and rejilla[nr, nc] == 0:
            yield (nr, nc)


def reconstruir(padres, meta):
    camino, actual = [], meta
    while actual is not None:
        camino.append(actual)
        actual = padres[actual]
    return camino[::-1]
```

```
def bfs(rejilla, inicio, meta, trace=False):
    t0 = time.perf_counter()
    frontera = deque([inicio])
    padres = {inicio: None}
    orden, hist = [], []
    max_frontera = 1
    while frontera:
        max_frontera = max(max_frontera, len(frontera))
        actual = frontera.popleft()
        orden.append(actual)
        if actual != meta:
            for v in vecinos(rejilla, actual):
                if v not in padres:
                    padres[v] = actual
                    frontera.append(v)
        if trace:
            hist.append(list(frontera))   # foto de la frontera tras cada expansión
        if actual == meta:
            break
    dt = time.perf_counter() - t0
    camino = reconstruir(padres, meta) if meta in padres else []
    return dict(nombre="BFS", camino=camino, expandidos=len(orden), orden=orden, hist=hist,
                tiempo_ms=dt * 1000, max_frontera=max_frontera, costo=len(camino) - 1)


def dfs(rejilla, inicio, meta, trace=False):
    t0 = time.perf_counter()
    frontera = [(inicio, None)]  # pila de (estado, padre)
    padres = {}
    orden, hist = [], []
    max_frontera = 1
    while frontera:
        max_frontera = max(max_frontera, len(frontera))
        actual, padre = frontera.pop()
        if actual in padres:
            continue
        padres[actual] = padre
        orden.append(actual)
        if actual != meta:
            # invertido para que el primer movimiento sea el que se explora primero
            for v in reversed(list(vecinos(rejilla, actual))):
                if v not in padres:
                    frontera.append((v, actual))
        if trace:
            hist.append(list({e for e, _ in frontera if e not in padres}))
        if actual == meta:
            break
    dt = time.perf_counter() - t0
    camino = reconstruir(padres, meta) if meta in padres else []
    return dict(nombre="DFS", camino=camino, expandidos=len(orden), orden=orden, hist=hist,
                tiempo_ms=dt * 1000, max_frontera=max_frontera, costo=len(camino) - 1)


def a_grafo(rejilla):
    """Representación explícita del espacio de estados como grafo NetworkX."""
    G = nx.Graph()
    for r in range(rejilla.shape[0]):
        for c in range(rejilla.shape[1]):
            if rejilla[r, c] == 0:
                G.add_node((r, c))
                for v in vecinos(rejilla, (r, c)):
                    G.add_edge((r, c), v)
    return G


def medir(func, rejilla, inicio, meta, repeticiones=200):
    """Promedia el tiempo de ejecución para reducir ruido."""
    res = func(rejilla, inicio, meta)
    ts = []
    for _ in range(repeticiones):
        ts.append(func(rejilla, inicio, meta)["tiempo_ms"])
    res["tiempo_ms"] = float(np.mean(ts))
    return res


# ------------------------------ Gráficas -----------------------------------
AZUL, VERDE, GRIS = "#1F4E9E", "#2E9E5B", "#D9DEE7"


def dibujar_laberinto(ax, rejilla, res, inicio, meta, titulo):
    cmap = ListedColormap(["#FFFFFF", "#2B2F3A"])
    ax.imshow(rejilla, cmap=cmap)
    # celdas expandidas (exploración)
    for (r, c) in res["orden"]:
        ax.add_patch(plt.Rectangle((c - 0.5, r - 0.5), 1, 1, color=GRIS, zorder=1))
    # camino solución
    if res["camino"]:
```

```
        ys, xs = zip(*res["camino"])
        ax.plot(xs, ys, color=AZUL, lw=3, zorder=3)
    ax.scatter(inicio[1], inicio[0], c=VERDE, s=140, zorder=4, label="Inicio")
    ax.scatter(meta[1], meta[0], c="#D1342F", s=140, marker="*", zorder=4, label="Meta")
    ax.set_title(f"{titulo}\nexpandidos={res['expandidos']}  |  longitud={res['costo']}", fontsize=10)
    ax.set_xticks([]); ax.set_yticks([])


def figuras(rejilla, inicio, meta, rb, rd, G):
    fig, axs = plt.subplots(1, 2, figsize=(11, 5))
    dibujar_laberinto(axs[0], rejilla, rb, inicio, meta, "BFS (cola FIFO)")
    dibujar_laberinto(axs[1], rejilla, rd, inicio, meta, "DFS (pila LIFO)")
    axs[0].legend(loc="lower left", fontsize=8)
    fig.tight_layout()
    fig.savefig(f"{SALIDA}/fig1_laberinto_bfs_dfs.png", dpi=170)
    plt.close(fig)

    # Grafo del espacio de estados con camino de BFS resaltado
    fig, ax = plt.subplots(figsize=(8, 5.5))
    pos = {n: (n[1], -n[0]) for n in G.nodes}
    nx.draw_networkx_edges(G, pos, ax=ax, edge_color="#B8C0CF", width=1)
    nx.draw_networkx_nodes(G, pos, ax=ax, node_size=28, node_color="#8FA6CF")
    cam = rb["camino"]
    nx.draw_networkx_edges(G, pos, ax=ax, edgelist=list(zip(cam, cam[1:])), edge_color=AZUL, width=3)
    nx.draw_networkx_nodes(G, pos, nodelist=[inicio], node_color=VERDE, node_size=120, ax=ax)
    nx.draw_networkx_nodes(G, pos, nodelist=[meta], node_color="#D1342F", node_size=120, node_shape="*", ax=ax)
    ax.set_title(f"Grafo del espacio de estados (|V|={G.number_of_nodes()}, |E|={G.number_of_edges()})")
    ax.axis("off")
    fig.tight_layout()
    fig.savefig(f"{SALIDA}/fig2_grafo_estados.png", dpi=170)
    plt.close(fig)

    # Comparación de métricas
    etiquetas = ["Nodos expandidos", "Longitud del camino", "Frontera máx.", "Tiempo (ms ×100)"]
    vb = [rb["expandidos"], rb["costo"], rb["max_frontera"], rb["tiempo_ms"] * 100]
    vd = [rd["expandidos"], rd["costo"], rd["max_frontera"], rd["tiempo_ms"] * 100]
    x = np.arange(len(etiquetas)); w = 0.38
    fig, ax = plt.subplots(figsize=(8.5, 4.6))
    b1 = ax.bar(x - w / 2, vb, w, label="BFS", color=AZUL)
    b2 = ax.bar(x + w / 2, vd, w, label="DFS", color=VERDE)
    ax.bar_label(b1, fmt="%.0f", fontsize=8); ax.bar_label(b2, fmt="%.0f", fontsize=8)
    ax.set_xticks(x); ax.set_xticklabels(etiquetas, fontsize=9)
    ax.set_title("Comparación BFS vs DFS en el laberinto principal")
    ax.legend(); ax.spines[["top", "right"]].set_visible(False)
    fig.tight_layout()
    fig.savefig(f"{SALIDA}/fig3_comparacion.png", dpi=170)
    plt.close(fig)


# ---------------------- Experimento de escalabilidad ----------------------
def laberinto_aleatorio(n, densidad, semilla):
    rng = np.random.default_rng(semilla)
    while True:
        rej = (rng.random((n, n)) < densidad).astype(int)
        rej[0, 0] = 0; rej[n - 1, n - 1] = 0
        ini, met = (0, 0), (n - 1, n - 1)
        if bfs(rej, ini, met)["camino"]:
            return rej, ini, met


def escalabilidad():
    tamanos = [10, 20, 30, 40, 50, 60]
    filas = []
    for n in tamanos:
        eb, ed, cb, cd = [], [], [], []
        for s in range(8):
            rej, ini, met = laberinto_aleatorio(n, 0.25, 100 * n + s)
            rb, rd = bfs(rej, ini, met), dfs(rej, ini, met)
            eb.append(rb["expandidos"]); ed.append(rd["expandidos"])
            cb.append(rb["costo"]); cd.append(rd["costo"])
        filas.append(dict(n=n, bfs_exp=float(np.mean(eb)), dfs_exp=float(np.mean(ed)),
                          bfs_costo=float(np.mean(cb)), dfs_costo=float(np.mean(cd))))
    fig, axs = plt.subplots(1, 2, figsize=(10, 4.2))
    ns = [f["n"] for f in filas]
    axs[0].plot(ns, [f["bfs_exp"] for f in filas], "o-", color=AZUL, label="BFS")
    axs[0].plot(ns, [f["dfs_exp"] for f in filas], "s-", color=VERDE, label="DFS")
    axs[0].set_title("Nodos expandidos vs tamaño (n×n)"); axs[0].set_xlabel("n"); axs[0].legend()
    axs[1].plot(ns, [f["bfs_costo"] for f in filas], "o-", color=AZUL, label="BFS")
    axs[1].plot(ns, [f["dfs_costo"] for f in filas], "s-", color=VERDE, label="DFS")
    axs[1].set_title("Longitud del camino vs tamaño (n×n)"); axs[1].set_xlabel("n"); axs[1].legend()
    for a in axs:
        a.spines[["top", "right"]].set_visible(False)
    fig.tight_layout()
    fig.savefig(f"{SALIDA}/fig4_escalabilidad.png", dpi=170)
    plt.close(fig)
    return filas
```

```
# ------------------------------ Simulación ----------------------------------
def _hex(c):
    c = c.lstrip("#")
    return [int(c[i:i + 2], 16) for i in (0, 2, 4)]


C_LIBRE, C_PARED = _hex("#FFFFFF"), _hex("#2B2F3A")
C_VISITADO, C_FRONTERA = _hex("#BFD0EE"), _hex("#F2B84B")
C_ACTUAL, C_CAMINO = _hex("#D1342F"), _hex("#1F4E9E")


def simular(rejilla, inicio, meta, velocidad=1.0, mostrar=True, archivo_gif=None):
    """Animación paso a paso: BFS (izq.) y DFS (der.) resolviendo el mismo laberinto.
    Amarillo = frontera (por explorar) · azul claro = visitado · rojo = nodo que se expande ·
    azul fuerte = camino solución."""
    rb = bfs(rejilla, inicio, meta, trace=True)
    rd = dfs(rejilla, inicio, meta, trace=True)
    res = [rb, rd]
    pasos = max(len(rb["orden"]), len(rd["orden"]))
    largo_camino = max(len(rb["camino"]), len(rd["camino"]))
    espera = 12
    total = pasos + largo_camino + espera

    fig, axs = plt.subplots(1, 2, figsize=(11, 5.8))
    fig.suptitle("Simulación de búsqueda ciega en un laberinto", fontsize=13, fontweight="bold", color=AZUL)
    ims, lineas, textos = [], [], []
    for ax, r in zip(axs, res):
        im = ax.imshow(np.zeros((*rejilla.shape, 3), dtype=np.uint8), interpolation="nearest")
        ln, = ax.plot([], [], color=AZUL, lw=3.2, zorder=3)
        ax.scatter(inicio[1], inicio[0], c=VERDE, s=150, zorder=4)
        ax.scatter(meta[1], meta[0], c="#D1342F", s=170, marker="*", zorder=4)
        ax.set_xticks([]); ax.set_yticks([])
        textos.append(ax.set_title("", fontsize=10))
        ims.append(im); lineas.append(ln)
    leyenda = [plt.Rectangle((0, 0), 1, 1, color=np.array(c) / 255)
               for c in (C_FRONTERA, C_VISITADO, C_ACTUAL, C_CAMINO)]
    fig.legend(leyenda, ["Frontera", "Visitado", "Expandiendo", "Camino"], loc="lower center", ncol=4,
               fontsize=9, frameon=False)
    fig.tight_layout(rect=(0, 0.06, 1, 0.95))

    def imagen(r, k):
        img = np.empty((*rejilla.shape, 3), dtype=np.uint8)
        img[:] = C_LIBRE
        img[rejilla == 1] = C_PARED
        for (a, b) in r["orden"][:k]:
            img[a, b] = C_VISITADO
        if 0 < k <= len(r["hist"]):
            for (a, b) in r["hist"][k - 1]:
                img[a, b] = C_FRONTERA
        if 0 < k <= len(r["orden"]):
            a, b = r["orden"][k - 1]
            img[a, b] = C_ACTUAL
        return img

    def cuadro(f):
        for i, r in enumerate(res):
            n = len(r["orden"])
            k = min(f, n)
            ims[i].set_data(imagen(r, k))
            fase = "buscando" if f < n else "meta encontrada"
            front = len(r["hist"][k - 1]) if 0 < k <= len(r["hist"]) else 1
            if f >= pasos:
                p = min(f - pasos + 1, len(r["camino"]))
                ys, xs = zip(*r["camino"][:p])
                lineas[i].set_data(xs, ys)
            else:
                lineas[i].set_data([], [])
            textos[i].set_text(f"{r['nombre']} · {fase}\nexpandidos={k}  frontera={front}  "
                               f"longitud camino={r['costo'] if f >= n else '?'}")
        return ims + lineas + textos

    anim = FuncAnimation(fig, cuadro, frames=total, interval=140 / velocidad, blit=False, repeat=True)

    estado = {"pausa": False}

    def al_clic(_):
        if estado["pausa"]:
            anim.resume()
        else:
            anim.pause()
        estado["pausa"] = not estado["pausa"]

    fig.canvas.mpl_connect("button_press_event", al_clic)

    if archivo_gif:
        anim.save(archivo_gif, writer=PillowWriter(fps=max(1, round(7 * velocidad))), dpi=90)
    # fotogramas fijos para el reporte (inicio, mitad, final de la búsqueda, camino)
    marcas = [3, pasos // 3, (2 * pasos) // 3, total - 1]
```

```
    fig2, axs2 = plt.subplots(2, 2, figsize=(11, 7.4))
    for ax2, f in zip(axs2.ravel(), marcas):
        cuadro(f)
        fig.canvas.draw()
        foto = np.asarray(fig.canvas.buffer_rgba())[..., :3]
        no_blanco = np.where(foto.min(axis=2) < 250)
        foto = foto[no_blanco[0].min():no_blanco[0].max() + 1, no_blanco[1].min():no_blanco[1].max() + 1]
        ax2.imshow(foto)
        ax2.axis("off")
    fig2.tight_layout()
    fig2.savefig(f"{SALIDA}/fig5_fotogramas_simulacion.png", dpi=150)
    plt.close(fig2)
    if mostrar and plt.get_backend().lower() not in ("agg", "pdf", "svg", "cairo", "ps", "template"):
        plt.show()
    plt.close(fig)
    return anim


def main():
    ap = argparse.ArgumentParser(description="BFS vs DFS con simulación animada")
    ap.add_argument("--sin-ventana", action="store_true", help="no abrir la ventana; solo guardar el GIF")
    ap.add_argument("--velocidad", type=float, default=1.0, help="velocidad de la animación (1 = normal)")
    args = ap.parse_args()
    rejilla, inicio, meta = cargar(LABERINTO)
    G = a_grafo(rejilla)
    rb = medir(bfs, rejilla, inicio, meta)
    rd = medir(dfs, rejilla, inicio, meta)
    figuras(rejilla, inicio, meta, rb, rd, G)
    esc = escalabilidad()
    simular(rejilla, inicio, meta, velocidad=args.velocidad, mostrar=not args.sin_ventana,
            archivo_gif=f"{SALIDA}/simulacion_bfs_dfs.gif")

    resumen = {
        "laberinto": {"filas": rejilla.shape[0], "columnas": rejilla.shape[1],
                      "nodos_libres": G.number_of_nodes(), "aristas": G.number_of_edges(),
                      "inicio": inicio, "meta": meta},
        "BFS": {k: rb[k] for k in ("expandidos", "costo", "max_frontera", "tiempo_ms")},
        "DFS": {k: rd[k] for k in ("expandidos", "costo", "max_frontera", "tiempo_ms")},
        "escalabilidad": esc,
    }
    with open(f"{SALIDA}/metricas.json", "w", encoding="utf-8") as f:
        json.dump(resumen, f, indent=2, ensure_ascii=False)

    print(f"{'Algoritmo':<10}{'Expandidos':>12}{'Longitud':>10}{'Frontera':>10}{'ms':>9}")
    for r in (rb, rd):
        print(f"{r['nombre']:<10}{r['expandidos']:>12}{r['costo']:>10}{r['max_frontera']:>10}{r['tiempo_ms']:>9.3f}")


if __name__ == "__main__":
    main()
```